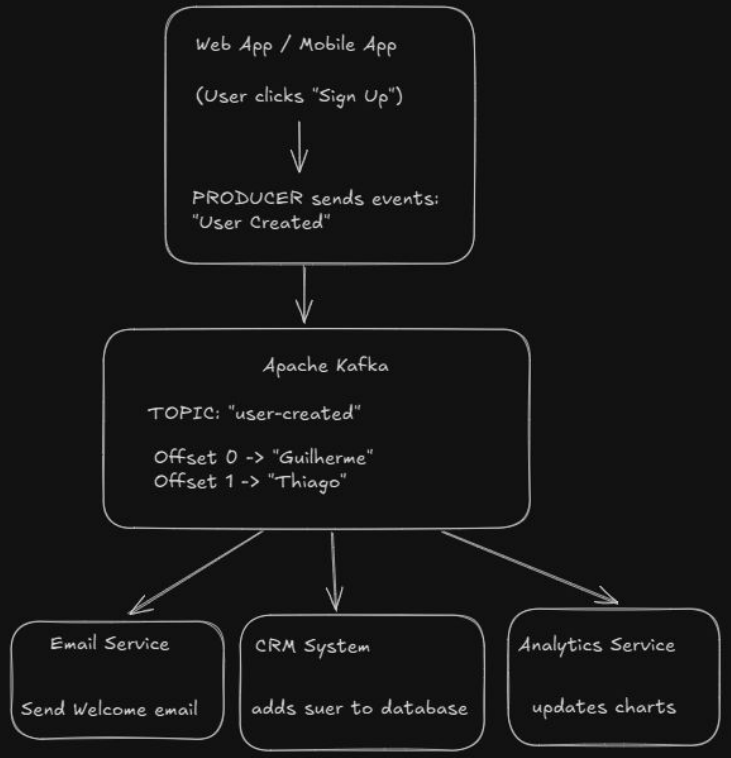
Kafka

Apache Kafka - distributed event streaming platform

Real time data streams at scale

Overview


Web App / Mobile App
(User clicks "Sign Up")
PRODUCER sends events:
"User Created"
Apache Kafka
TOPIC: "user-created"
Offset 0 -> "Guilherme"
Offset 1 -> "Thiago"
Email Service
Send Welcome email
CRM System
adds suer to database
Analytics Service
updates charts

Kafka - Timeline

2010 – Kafka created at LinkedIn.

2011 – Open-sourced and joined Apache Incubator.

2012 – Became a top-level Apache project.

2014 – Founders launched Confluent.

2015–2017 – Added Kafka Connect and Kafka Streams.

2017–2021 – Major scalability upgrades.

2021–2024 – Moved toward ZooKeeper-free mode (KRaft).

2025 – Kafka 4.0 new "Queues for Kafka"

Use Cases

- Distributed logging
- Stream processing
- Pub-sub messaging

Pub/Sub: Example

```
class KafkaBroker
{
    public event Action<string>? MessagePublished;
    public void Publish(string message)
    {
        Console.WriteLine($"[Broker] Publishing message: {message}");
        MessagePublished?.Invoke(message);
    }
}
```

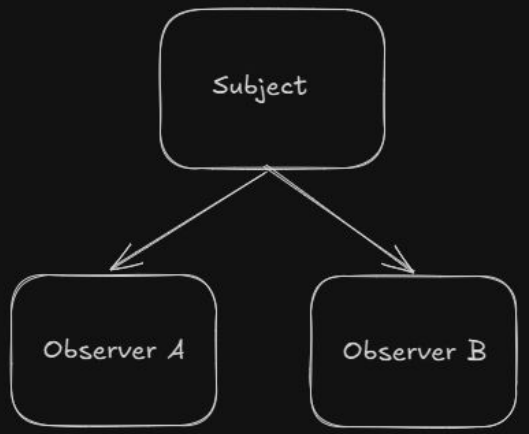
Pub/Sub: Example

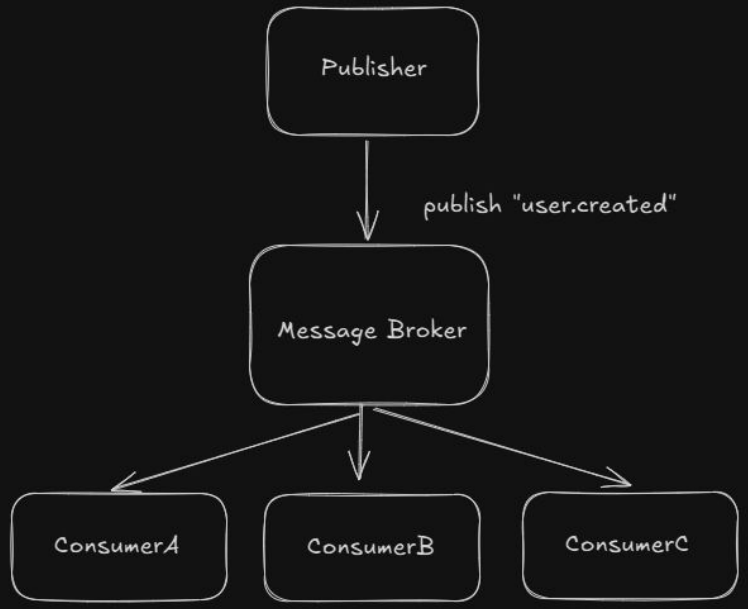
```
class Program
{
    static void Main()
    {
        var broker = new KafkaBroker();

        broker.MessagePublished += msg => Console.WriteLine($"[Consumer-A] Received: {msg}");
        broker.MessagePublished += msg => Console.WriteLine($"[Consumer-B] Processing: {msg}");
        broker.MessagePublished += msg => Console.WriteLine($"[Consumer-C] Stored: {msg}");

        broker.Publish("user.created: Robson");
        broker.Publish("user.created: Cesar");
        broker.Publish("user.created: Jorge");
    }
}
```

Pub/Sub: Observer Pattern vs Pub/Sub





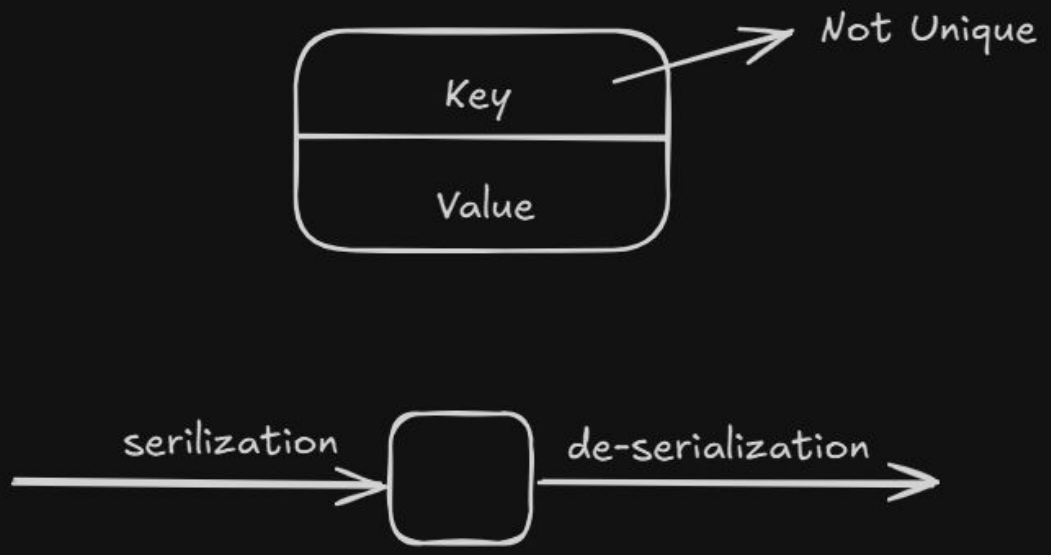
Producer - Consumer

A **Producer** is a client that **sends messages** to Kafka.
It chooses the **topic** and **partition** where each message should go.
Producers are responsible for delivering data reliably, using acknowledgments (`acks`) to confirm success.

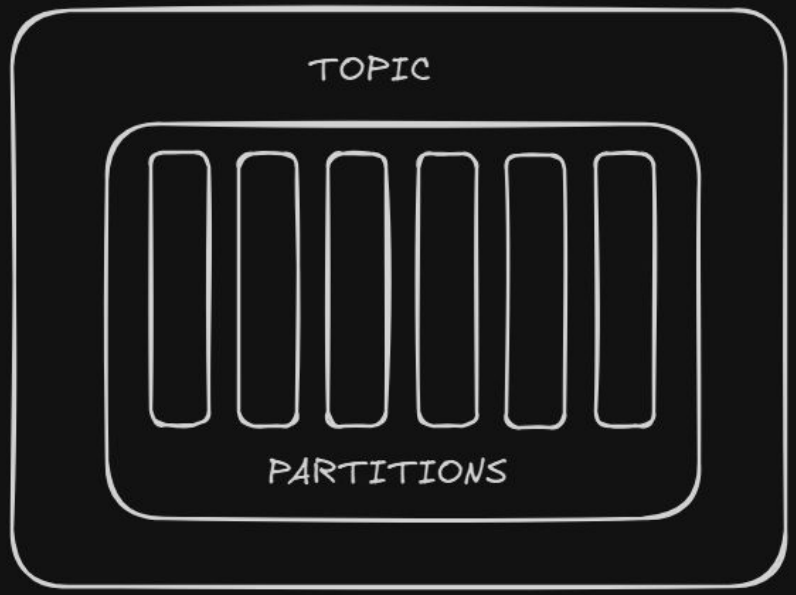
Producer - Consumer

A **Consumer** is a client that **reads messages** from Kafka.
It subscribes to one or more topics and processes the data in order.
Consumers work as part of **consumer groups**, which share the load of reading messages from different partitions.

Message



Cluster



Topics

Named container for similar events

Systems contains lots of topics

Can duplicate data between topics

Durable logs of events

Append only

Can only seek by offset, not indexed

Events are immutable

Topics are durable

Retention period is configurable

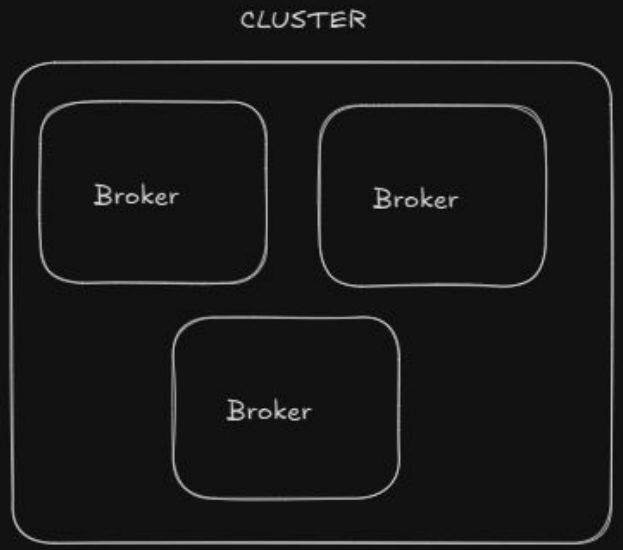
Partitioning

Key is null -> round robin -> uniform (Not random)

Key defined -> order -> {customer_id} (Key based)

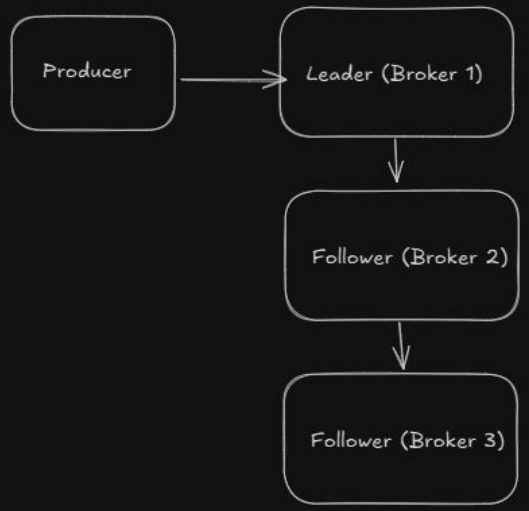
Broker

Manager partitions
Handle write and read requests
Manage replication of partitions
Intentionally very simple



Replication

If Broker 1 fails → Broker 2 automatically becomes new leader.



Replication

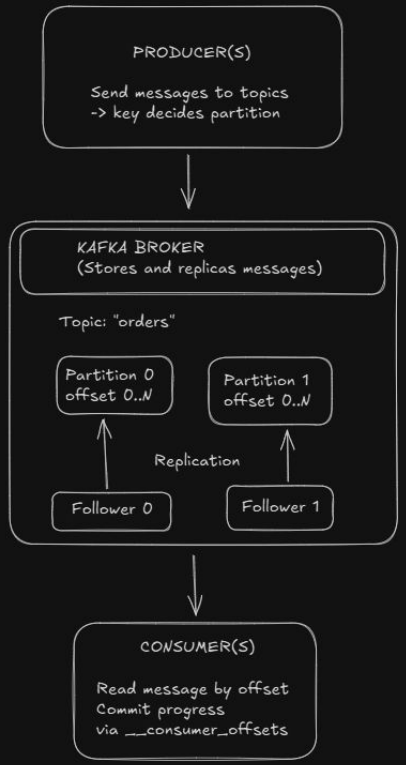
Copies of data for fault tolerance

One lead portion and N-1 followers

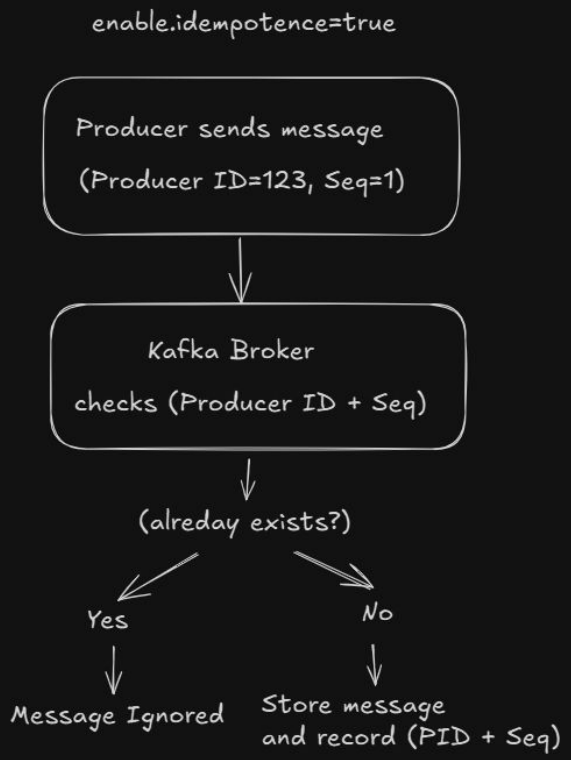
An invisible process to most developers

```
Topic: create-user
Partitions: 3
Replication factor: 3
```

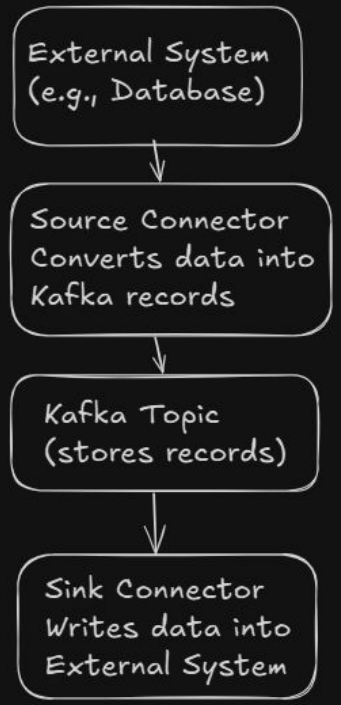
Kafka Overview



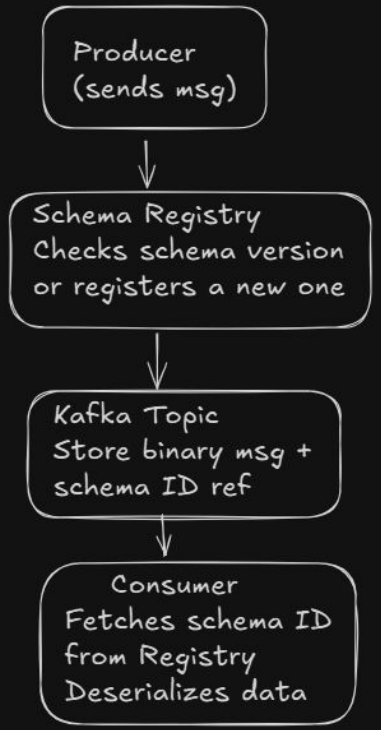
Idempotence



Kafka Connect



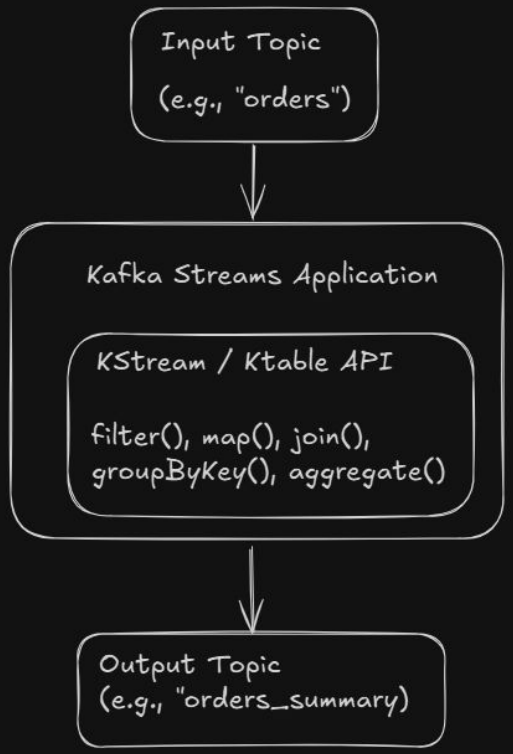
Schema Registry



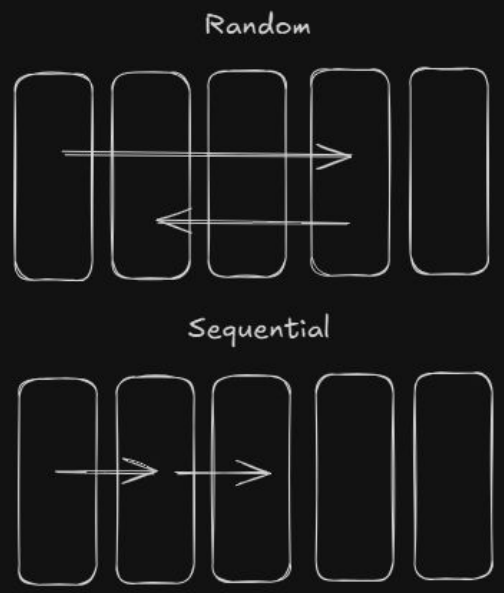

```
{
  "type": "record",
  "namespace": "com.example",
  "name": "User",
  "fields": [
    { "name": "id", "type": "int" },
    { "name": "name", "type": "string" },
    { "name": "email", "type": ["null", "string"], "default": null },
    { "name": "age", "type": ["null", "int"], "default": null },
    { "name": "created_at", "type": "string", "logicalType": "timestamp-millis" }
  ]
}
```

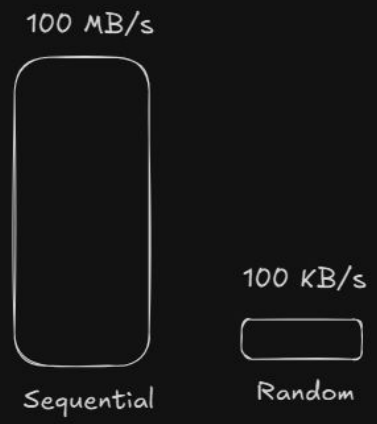
Supported Formats
- Json Schema
- Avro
- Protocol Buffer

Kafka Streams

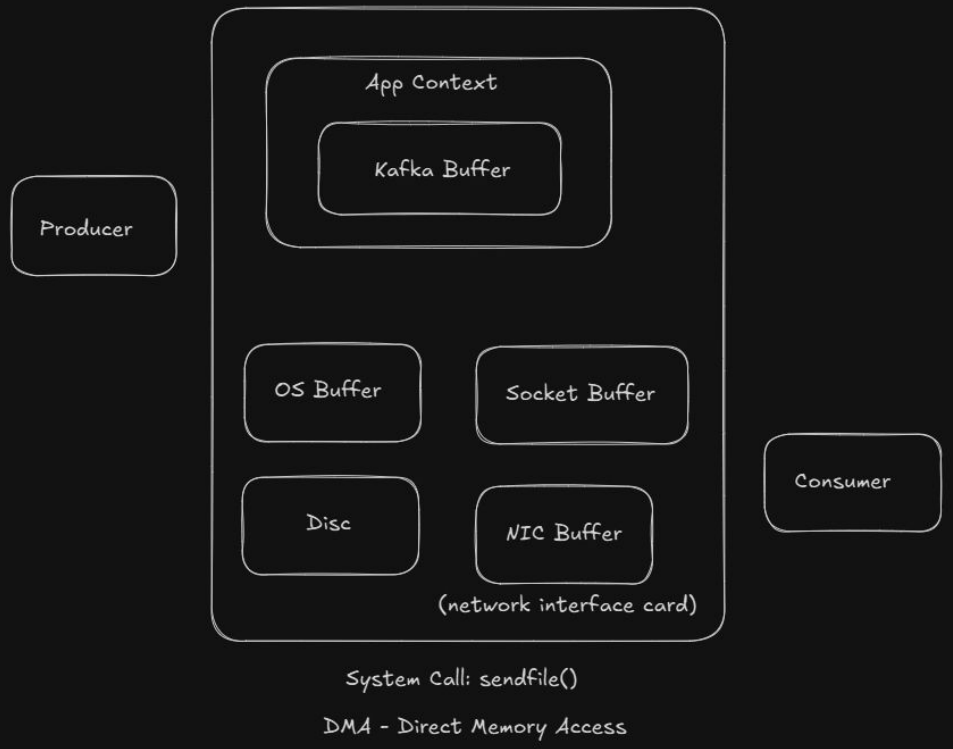
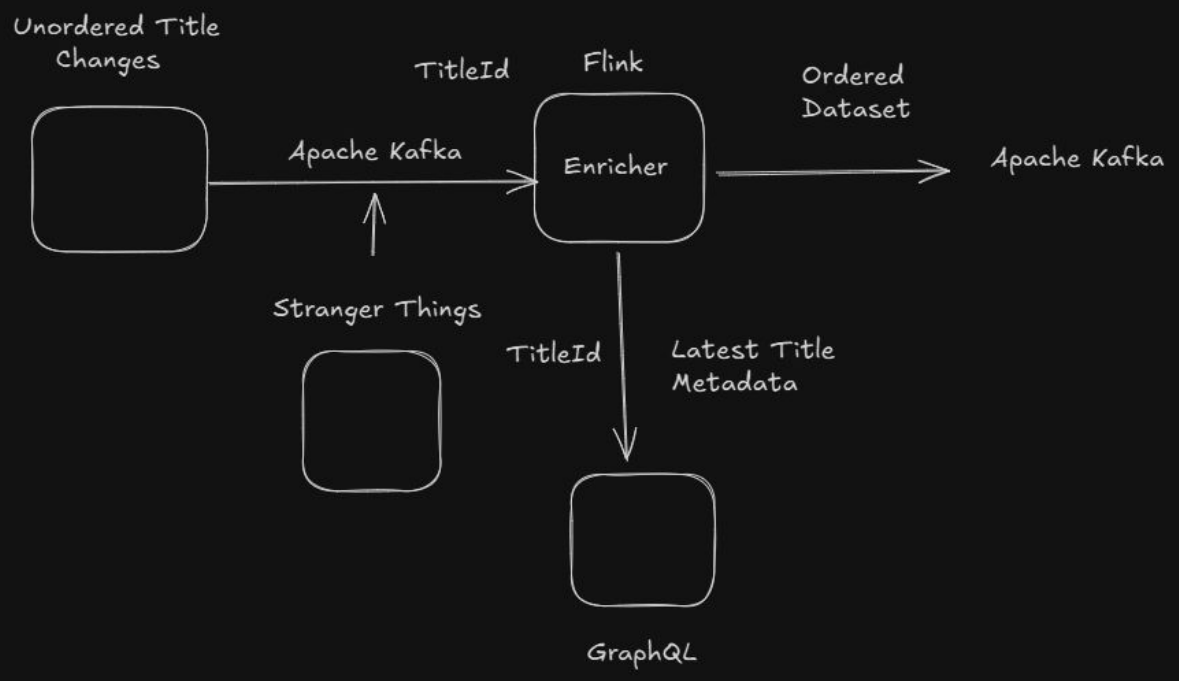


Kafka Performance: sequential





Kafka Performance: zero copy



Kafka - Netflix Use Case